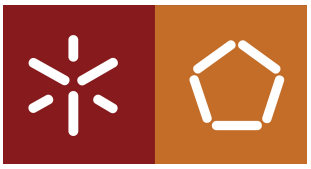

Sensing with L-SNMPvS

Practical Assignment

Author:
Amneh Khaled Al-Issa
Student Number: E12305

University of Minho
Course: Network Management & Security

Submission Date: 11 June 2025

Contents

1 Introduction

This project is a prototype implementation of the **Lightweight Simple Network Management Protocol for Sensors (L-SNMPvS)**. The aim is to simulate a system where remote sensor devices can be monitored and managed over a network using a simplified SNMP-like approach. The system is made up of two main parts:

- An **agent**, which holds and updates sensor-related data.
- A **manager**, which sends requests to the agent and processes its responses.

2 Design Strategies, Decisions, and Technologies

2.1 Protocol and Communication

The **L-SNMPvS** protocol, as described in the assignment, is used for all communication between the manager and agent. This protocol is inspired by SNMP but is simplified and extended to support additional features. `UDP` sockets are used for transport, providing lightweight, connectionless communication suitable for sensor networks.

2.2 Code Architecture

The code is organized across multiple files to keep things clear and maintainable. First, we have the `PDU-header.hpp` file, which contains shared protocol definitions and the logic for encoding and decoding messages. Then, the `MIB-header.hpp` file contains definitions and structures related to the Management Information Base (MIB), which helps model the device and sensor data.

The agent and manager are implemented as separate programs, each with its own responsibilities. The agent maintains a lightweight MIB (L-MIB) that represents both device and sensor objects. Meanwhile, the manager offers a command-line interface that allows users to send requests, receive responses, and display the relevant sensor and device information in a user-friendly way.

2.3 Technology Choices

The implementation is written in `C++` for portability and efficiency, using standard libraries for networking (POSIX sockets), threading, random number generation (for virtual sensor values), and time management (for timestamps). `OpenSSL` is used for `SHA-256` hashing to meet the message integrity requirement. STL containers such as `std::vector` and `std::map` are used for efficient data management.

3 Main Components and Implementation

3.1 PDU Definition & Encoding/Decoding (`PDU-header.hpp`)

The file `PDU-header.hpp` contains all protocol definitions and encoding/decoding logic shared by the agent and manager. Key elements include:

- **Standard library includes:** All required C++ and POSIX headers for networking, threading, cryptography (OpenSSL), and data structures.
- **Structures:**
  - `LSNMP_PDU`: Main protocol data unit, with fields for tag, message type, timestamp, message ID, and lists for IIDs, values, timestamps, and error codes.
  - `LSNMP_IID`: Represents instance identifiers.
  - `LSNMP_Value`: Supports integer, string, timestamp, and IID types.
  - `LSNMP_Timestamp`: Used for absolute and relative time values.
- **Enums:** Definitions for message types (`LSNMP_Type`), error codes (`LSNMP_ErrorCode`), value types (`LSNMP_ValueType`), and data type encoding (`LSNMP_DataType`).
- **Security constants:** Agent/manager/broadcast IDs, security model IDs, and a shared secret key. Implements XOR-based encryption and `SHA-256` hashing for message integrity.
- **Encoding/decoding helpers:** Inline functions that convert integers, timestamps, IIDs, and values into a sequence of bytes (and back).
- **List encoding:** Templated functions that handle storing and reading lists of values in byte format.
- **Main encode/decode:**
  - `encodePDU()`: Converts an `LSNMP_PDU` into a byte array so it can be sent over the network.
  - `decodePDU()`: Reconstructs an `LSNMP_PDU` from the received byte data.
  - `encodeSecureMessage()` and `decodeSecureMessage()`: Add or remove security wrapping to messages, based on the protocol requirements.

3.2 MIB Structure (`MIB-header.hpp`)

The file `MIB-header.hpp` defines the **L-SNMPvS** Management Information Base (MIB) class along with related logic for managing device and sensor data on the agent side. The main components are within the main class (`LSNMP_MIB`) and they include:

- **Initializer:** Initializes all device and sensor objects. It creates two devices along with their corresponding sensors. It uses static values.
- **Data Storage (Maps):** All device and sensor data is stored using C++ `'std::map'` containers, which allow efficient lookup and update by instance identifier (IID).
  - `deviceData`: Stores all device-related values indexed by IID.
  - `sensorsData`: Stores all sensor-related values indexed by IID.
  - `deviceBootTimes`: Tracks the boot time for each device.
- **Functions for Query/Update:**
  - `has()`: Checks if a given IID exists in the MIB.
  - `get()`: Retrieves the value for a given IID, including special processing for time and uptime fields.
  - `set()`: Updates writable fields (e.g., beacon rate, date/time, reset, sensor sampling rate) with appropriate type and value checks.
- **Object Update Time Tracking:**
  - `lastUpdateTimes`: A map to record the timestamp of the last update for each IID.
  - `recordUpdateTime()`: A function that updates the last update time for a specific IID.
  - `getLastUpdateAge()`: A function that calculates how much time has passed since an IID was last updated, returning success or failure depending on whether the IID exists in the `lastUpdateTimes` map.
- **Getter functions:**
  - `getAllDeviceData()`, `getAllSensorData()`: Return all device or sensor data for summary or debugging purposes.
- **Internal Helpers:**
  - `addDeviceObject()`, `addSensorRow()`: Used by the constructor to add device and sensor entries to the MIB.
  - `updateSampleValue()`, `updateLastSamplingTime()`: For internal updates to sensor values and timestamps.

3.3 Agent Logic (`agent.cpp`)

The `agent.cpp` file contains the code for the **L-SNMPvS** agent. The agent manages sensor data, communicates over the network using UDP, and sends regular broadcast messages called **beacons**. The main parts are:

- **Global Variables:**
  - `agentRebootTime`: Stores the time when the agent started. This helps calculate how long the agent has been running. And it was also used to calculate the age of the objects.
  - `running`: A flag used to control the sensor update thread safely, which is responsible for generating random sensor values.
  - `mibMutex`: A lock to make sure only one thread accesses the MIB data at a time.
- **Main Functions:**
  - `getAgentUptime()`: Calculates how long the agent has been running since it started.
  - `getCurrentTimestamp()`: Gets the current time to use in the messages.
  - `sensorUpdateThread()`: Runs in the background to update sensor values regularly using random numbers and updates their timestamps. It's a way to simulate how real life sensors work, and to make the data 'not so static'.
  - `sendBeacon()`: Sends a broadcast message to all managers, sharing key device information at regular intervals.
- **Main Loop:**
  - Sets up a UDP socket for communication, allows broadcast messages, and listens on the correct port.
  - Starts the sensor update thread.
  - Sends beacon messages regularly.
  - Listens for incoming messages from managers, checks security, and processes their requests.
  - Handles two types of requests:
    - `GET`: Looks up requested data in the MIB and returns the values and timestamps.
    - `SET`: Updates only the allowed fields in the MIB, and it handles special commands like reset, and sends back the updated data or error messages.

* The current agent implementation only supports beacons; other message types like Notifications or Traps I did not include.

```
while (true) {
    // ... receive message ...
    if (receivedPDU.type == GET_REQUEST)
        // ... lookup and respond ...
    else if (receivedPDU.type == SET_REQUEST)
        // ... update and respond ... }
```

  - Encodes the response, applies security features, and sends it back to the manager.

- **Shutdown:**
  - Closes the UDP socket and stops the sensor update thread when the agent exits.

3.4 Manager Logic (`manager.cpp`)

The file `manager.cpp` implements the **L-SNMPvS** manager console, which lets users communicate with the agent, send requests, and display device and sensor information. The main parts are:

- **Global Variables:**
  - `lastRequest`, `lastResponse`: Store the most recent request and response PDUs for debugging and the `pdu` command.
- **Helper Functions:**
  - `parseIID()`: Converts user input strings into valid `LSNMP_IID` structures, supporting both full and short forms for devices and sensors.
  - `describeIID()`: Provides a readable description of an IID, mapping protocol fields to device or sensor names.
  - `describeError()`: Turns error codes into understandable messages for the user.
  - `printIID()`: Prints IIDs in a consistent format.
  - `getCurrentTimestamp()`: Generates the current timestamp for outgoing PDUs.
- **Request and Response Handling:**
  - `sendRequest()`: Sends a PDU request to the agent, receives and checks the response security, decodes the PDU, and prints the results.
  - `sendRequestAndGetResponse()`: Sends a request and returns the decoded response PDU (used for summary command).

- `printPDUInfo()`: Prints detailed PDU information for debugging.
- `printSummary()`: Sends batch requests to get and display a table summary of all devices and sensors.
- `handleBeacon()`: Processes and displays beacon notifications received from the agent.

- **Main Loop:**
  - Sets up a UDP socket and prepares the server address.
  - Shows a welcome message and available commands.
  - Waits for user input and processes commands such as:
    - `get`: Sends a GET request for one or more IIDs and prints the results.

```
if (cmd == "get") {
    std::vector<LSNMP_IID> iids;
    // ... parse IIDs from user input ...
    LSNMP_PDU request;
    request.type = GET_REQUEST;
    request.iidList = iids;
    // ... set timestamp, messageID ...
    sendRequest(request, sockfd, server_addr,
       server_len);
}
```

    - `set`: Sends a SET request for one or more IIDs and values (supports integers and timestamps).
    - `summary`: Prints a table summary of all devices and sensors.
    - `pdu`: Displays details of the last request and response PDUs.
    - `help`: Prints help and usage instructions.
    - `exit`: Exits the manager console.
  - Listens for beacon notifications from the agent and shows them when received.

4 Security Mechanisms

The L-SNMPvS implementation includes basic security features as required by the assignment. The main security parts are:

- **Secure Message Envelope:**
  - Each message is wrapped in a secure envelope with sender and receiver IDs, a security model ID, and the size of the secured data.
  - For regular (non-broadcast) messages, the security model ID is set to 128 (`SEC_MODEL_SIMPLE`).
  - Broadcast messages, like beacons, have security turned off (security model ID 0).

```
inline std::vector<uint8_t> encodeSecureMessage(
    const std::vector<uint8_t>& pdu,
    const std::vector<uint8_t>& senderID,
    const std::vector<uint8_t>& receiverID,
    uint8_t secModel = SEC_MODEL_SIMPLE)
{
    std::vector<uint8_t> buf;
    // ... add IDs, encrypt, append hash ...
    return buf;
}
```

- **Symmetric Key Encryption:**
  - A shared secret key (`SHARED_KEY`) is used to encrypt and decrypt messages.
  - Encryption is done by XOR'ing each message byte with the key bytes.
  - Only the agent and manager know this key, providing simple mutual authentication.
- **Message Integrity:**
  - After encryption, a `SHA-256` hash of the encrypted message is added to help detect tampering.
  - When a message is received, the hash is checked before decrypting.
- **Security Functions in Code:**
  - `encodeSecureMessage()`: Encrypts the PDU, adds the hash, and creates the secure envelope.
  - `decodeSecureMessage()`: Checks the hash, decrypts the message, and extracts the original PDU.

5 Testing and Demonstration

To test the system, I ran the agent and manager programs in two different terminals using WSL (Windows Subsystem for Linux). Both programs were compiled with the required libraries for security:

```
// In terminal 1
$ g++ agent.cpp -o agent -lssl -lcrypto

// In terminal 2
$ g++ manager.cpp -o manager -lssl -lcrypto
```

5.1 Testing Communication Between Agent and Manager

First, I tested basic communication. I ran both the agent and manager, then used the manager to send some commands (like get) to the agent. The agent received the messages, processed them, and sent back the correct responses.

- Below is a basic example:

```
amnah@Vos:~/assignment$ ./manager
Welcome to L-SNMPvS Manager Console (type 'exit' to quit)

Type 'help' for available commands.
Type 'summary' to get a brief overview of connected devices and sensors.
Type 'pdu' to see the last request and response PDU details.

> get 1.6.2
[Response for MSG-ID 1000]
  IID 1.6.2 => System Date and Time [Device 2] : 09:06:2025:23:31:00:102
  Error: 0 (No Error (Operation successful))
```

- Another example shows the manager resetting one of the devices:

First, we can check the uptime for the device (which we assume matches the agent uptime) using the 'summary' command to make sure it will reset later...

```
> summary

==================== L-SNMPvS Device & Sensor Summary ====================

Device #1 (Sensing Hub 1)
  ID: 00:11:22:33:44:55
  Uptime: 00:00:0000 00:06:12.310
  Date/Time: 09:06:2025 23:35:34.291
  Status: 1
  Sensors: 2
    ------------------------------------------------------------------------
    | # | Type         | Value | Min | Max | LastSampleTime          | Rate |
    ------------------------------------------------------------------------
    | 1 | temperature  | 0     | 246 | 40  | 00:00:0000 01:06:04.756 | 10   |
    | 2 | humidity     | 1     | 0   | 100 | 00:00:0000 01:06:04.756 | 10   |
    ------------------------------------------------------------------------
```

After the manager resets the first device, we can see the updated uptime below:

```
> set 1.9 1
[Response for MSG-ID 1018]
  IID 1.9.1 => Reset Flag [Device 1] : 1
  Error: 0 (No Error (Operation successful))

> summary

===================== L-SNMPvS Device & Sensor Summary ====================

Device #1 (Sensing Hub 1)
  ID: 00:11:22:33:44:55
  Uptime: 00:00:0000 00:00:04.251
```

- Other basic examples (without going into details):

- Invalid IID Handling:

```
> get 3.2
Invalid IID format. Use 1.3 or 1.3.1.0 etc.
Type 'help' to know how to query objects.
No valid IIDs provided.
```

- Read-Only Field Protection:

```
> set 1.7 3
[Response for MSG-ID 1008]
  IID 1.7.1 => Device Uptime [Device 1] : 0:00:03:14:699
  Error: 9 (Value Exists but is Read-Only (Object cannot be written to, or write not permitted))
```

- Handling Multiple Requests:

```
> get 1.3.1 2.7.1.2 1.5.2
[Response for MSG-ID 1001]
  IID 1.3.1 => Device Type [Device 1] : Sensing Hub 1
  Error: 0 (No Error (Operation successful))

  IID 2.7.1.2 => Sampling Rate [Device 1, Sensor 2] : 10
  Error: 0 (No Error (Operation successful))

  IID 1.5.2 => Sensor Count [Device 2] : 2
  Error: 0 (No Error (Operation successful))
```

- Setting Timestamp (and it continues counting from that point)

```
> get 1.6.2
[Response for MSG-ID 1004]
  IID 1.6.2 => System Date and Time [Device 2] : 10:06:2025:13:36:48:962
  Error: 0 (No Error (Operation successful))

> set 1.6.2 "11:06:2024:15:45:59:34:444"
[Response for MSG-ID 1005]
  IID 1.6.2 => System Date and Time [Device 2] : 11:06:2024:15:45:59:034
  Error: 0 (No Error (Operation successful))

> get 1.6.2
[Response for MSG-ID 1006]
  IID 1.6.2 => System Date and Time [Device 2] : 11:06:2024:16:46:02:217
  Error: 0 (No Error (Operation successful))
```

5.2 Testing Security Features

To check if encryption and message integrity were working correctly, I printed the raw message data (in hex format) before and after applying security. These debug prints were only used during testing and were commented out later to keep the terminal clean.

Example: When the manager sends a `'get 1.2'` command, we can look at the raw message (hexdump) from the agent.

- **Without Security:** The plain message is easy to read, for example, you can see text like LSNMPvS1, the message type, and even the MAC address clearly.

```
[Plain PDU hexdump]
4C 53 4E 4D 50 76 53 31 03 55 A8 05 60 32 C9 00 00 00 00 00 00 03 E8 00 01 41 01 02 00 01 00 01 20 00 11 30 30 3A 31
31 3A 32 32 3A 33 33 3A 34 34 3A 35 35 00 01 0E 0F 00 00 00 00 00 01 00
```

  - **Offset 0-7:** `4C 53 4E 4D 50 76 53 31` – Protocol tag: `LSNMPvS1`
  - **Offset 8:** `03` – Message type: RESPONSE
  - **Offset 9-12:** `4A C3 03 94` – Message ID
  - **Offset 13-20:** `32 C9 00 00 00 00 00 00` – Timestamp and other fields
  - **Offset 32-47:** `30 30 3A 31 31 3A ...` – MAC Address: `00:11:22:33:44:55`

- **With Security:** After adding encryption and hashing, you can no longer see readable values, which means the data is now protected.

```
[Secured Message hexdump]
41 47 45 4E 54 00 00 01 4D 41 4E 41 47 45 52 01 80 00 00 00 5F 3F 36 2D 3F 35 02 38 48 70 30 CB 77 05 46 A2 79 73 65
63 72 65 77 83 79 72 24 62 70 65 75 6B 78 53 65 72 42 55 4E 5A 48 49 57 51 48 56 47 51 4D 47 5F 56 47 65 75 65 76 73
65 63 72 65 75 6B F4 8D 4B B1 8E 58 88 A4 BE BC F0 2D BA 51 36 3D 7C B5 5E 70 46 50 CE F7 9D 26 E8 74 20 5B 89 89
```

  - **Offset 0-7:** `41 47 45 4E 54 00 00 01` – Sender ID: `AGENT...`
  - **Offset 8-15:** `4D 41 4E 41 47 45 52 01` – Receiver ID: `MANAGER...`
  - **Offset 16:** `80` – Security Model ID (128)
  - **Offset 17-20:** – Secured Data Size
  - **Offset 21–end:** – Encrypted payload + SHA-256 hash

Another way to test the security is to flip a bit in the secured data and see if the message is accepted or not. By adding this line to the agent code...

```
if (secured.size() > 21) secured[21] ^= 0xFF;
```

After doing this, if the manager sends any request, the user will see the following message:

```
[Manager] Security check failed or invalid message.
```

6 Conclusion and Future Improvements

Conclusion:

- This project successfully created a working version of the L-SNMPvS protocol, including secure communication, a modular MIB, and both agent and manager parts.
- The agent can simulate many devices and sensors, answer GET/SET requests, and the manager has a simple command-line interface to talk with the agent, get summaries, and check protocol messages.

Future Improvements:

- **User Interface:** The manager uses a basic command-line interface. A web interface would make it easier and nicer to use, but I didn't have enough time.
- **Sensor Values:** Sometimes the random sensor values are outside the expected range. This can be fixed by improving how the values are generated to stay within the limits.
- **Beacon Reception:** Currently, the agent sends beacons correctly, but the manager does not receive or process these notifications as intended. This may be due to UDP broadcast handling or socket configuration. All other core features work as required.
- **Error Handling:** Adding clearer error messages and better logging would help, especially for protocol and security problems.
- **Extensibility:** The system could add support for more device types, more sensor details, or more complex MIB structures.
- **Security:** The current security is basic and mainly for demonstration. Stronger encryption and better key management would be needed for real use.
- **Testing:** Adding more automated tests and checking unusual cases would make the system more reliable, especially for parsing and MIB access.

7 References

- Assignment specification and lecture notes, Network Management & Security, University of Minho, 2025.
- C++ Reference: `https://en.cppreference.com/`
- OpenSSL: `https://www.openssl.org/docs/`
- Stack Overflow